

# TDK POS / TDK 관리센터

재구축 가능한 통합 설계 · 구현 기준서

기준일: 2026 - 10 - 01

목적: 이 문서 한 권만으로 현재까지 설계 · 구현한 TDK 시스템의 구조, 화면, 데이터베이스, 인증, 결제, 장치 연동, 배포 방향과 아직 미완성인 부분을 파악하고 동일한 시스템을 다시 구축할 수 있도록 정리한 기준 문서입니다.

중요: 이 문서는 현재까지의 대화와 확정된 설계를 기준으로 작성했습니다. 실제 소스코드의 모든 파일 내용과 현재 NAS DB의 정확한 DDL을 직접 추출한 문서는 아니므로, 재구축 시에는 본 문서의 '확정'과 '예정/미완성' 표시를 구분해야 합니다.

# 목차

- 1. 시스템 목표와 전체 아키텍처
- 2. 개발 · 운영 환경
- 3. 프로젝트 구조와 실행
- 4. 데이터베이스 기본 원칙
- 5. 핵심 DB 모델
- 6. 직원 · 인증 정책
- 7. POS 메인 화면
- 8. 주문 · 테이블 · 일행 처리
- 9. 인원 · 할인 · 메뉴 UX
- 10. 결제 핵심 구조
- 11. 기타결제
- 12. 고객 및 고객결제 설계
- 13. TDK 관리센터
- 14. 판매내역 · 거래상세 · 결제 변경
- 15. 현금시재
- 16. 영수증 · 프린터 · Device Agent
- 17. 주방/KDS · QR · 재고 확장
- 18. NAS 중앙서버 · 배포
- 19. 공통 UI 규칙
- 20. 현재 완료/진행/미완성
- 21. 처음부터 다시 만드는 권장 순서
- 22. 검증 체크리스트
- 23. 주요 결정사항 요약

# 1. 시스템 목표와 전체 아키텍처

TDK는 제주 제주시 음식점 '탐동김치찌개'의 주문 · 테이블 · 결제 · 영수증 · 고객 · 관리 업무를 하나의 웹 기반 시스템으로 통합하는 프로젝트다. 기존 Access/MDB POS를 보존하는 프로젝트가 아니라 새로 정규화하여 만드는 시스템이다.

운영 역할은 세 부분으로 분리한다.

- <b>NAS 중앙서버</b>: Next.js 서버/API + MariaDB. 매장의 데이터와 인증을 중앙에서 관리한다.
- <b>POS PC</b>: 브라우저의 /pos를 사용한다. 주문 · 테이블 · 결제처럼 영업 중 빠른 작업에 집중한다. 프린터/금고/향후 카드단말기는 로컬 Device Agent가 담당한다.
- <b>업무용 PC</b>: 브라우저의 /admin을 사용한다. 고객 · 결제설정 · 매출 · 메뉴 · 재고 · 직원 · 설정 같은 상세 관리 기능을 담당한다.

권장 운영 형태

```
[ASUSTOR NAS] TDK - POS Next.js + MariaDB | <br/> ┌──────────┴──────────┐ <br/> [POS PC /pos]
[업무 PC /admin] <br/> | <br/> [localhost Device Agent → BIXOLON/금고/향후 결제단말기]
```

## 2. 개발 · 운영 환경

| 항목           | 현재 기준                                          |
|--------------|------------------------------------------------|
| 개발 PC        | Windows + VS Code + Codex                      |
| 워크스페이스       | D:\10_프로그래밍_소스\01_TDK_Web                      |
| Next.js 앱    | D:\10_프로그래밍_소스\01_TDK_Web\tdk - pos            |
| Device Agent | D:\10_프로그래밍_소스\01_TDK_Web\tdk - device - agent |
| Node         | v22.17.0                                       |
| Next.js      | 16.3.5                                         |
| ORM          | Drizzle ORM 0.44.7 / drizzle - kit 0.31.4      |
| DB 드라이버      | mysql2 3.24.4                                  |
| DB           | MariaDB 10.11.8, DB명 TDK_POS                   |
| NAS DB 주소    | 192.168.0.83 (개발 당시 LAN 기준)                    |
| 현재 POS 모니터   | 24인치 1920×1080                                 |
| 프린터          | BIXOLON SRP - 350III USB                       |
| NAS          | ASUSTOR AS5404T, Docker 사용                     |

개발 실행

```
cd /d "D:\10_프로그래밍_소스\01_TDK_Web\tdk - pos"<br/>npm run dev
```

개발 PC에서는 Windows 자동시작보다 수동 npm run dev를 권장한다. 운영 매장에서는 NAS의 Docker 서버를 상시 운영하고 POS PC에서는 브라우저와 Device Agent만 자동시작하는 방향이다.

## 3. 프로젝트 구조와 실행

- 하나의 Next.js 프로젝트 안에 /pos, /admin, 향후 /kitchen을 둔다. 별도 관리센터 프로젝트를 만들지 않는다.

- /pos는 터치 중심 영업 UI, /admin은 마우스 · 키보드 중심 업무 UI로 디자인을 분리한다.
- MariaDB는 NAS의 기존 서비스를 유지하고, Next.js를 Docker 컨테이너로 올려 NAS MariaDB에 연결한다.
- 개발은 업무 PC에서 하고 검증된 빌드만 NAS로 배포한다. NAS에서 직접 소스를 수정하지 않는다.
- GitHub private 저장소 + .env 제외 + CI build 검증을 권장한다.

## 4. 데이터베이스 기본 원칙

- 새 정규화 DB를 사용하며 기존 MDB 구조를 억지로 보존하지 않는다.
- 테이블명은 복수형, 컬럼은 snake\_case, PK 명시, 시간은 \*\_at, boolean은 is\_\* 규칙을 기본으로 한다.
- 주문 상세를 삭제하여 이력을 없애지 않는다. 취소 · 환불 · 변경은 상태와 역거래로 감사 이력을 남긴다.
- 화면에 보이는 합계 캐시는 사용할 수 있지만 상세 원장과 트랜잭션으로 일관성을 유지한다.
- Drizzle pull/introspection은 안정적이지 않아 src/db/schema.ts를 수동 유지한다. 구조 변경은 phpMyAdmin에서 검토한 정확한 SQL을 직접 실행하는 방식을 우선한다.

## 5. 핵심 DB 모델

| 테이블/개념                  | 역할                                             |
|-------------------------|------------------------------------------------|
| dining_tables           | 물리 테이블 정의. 기본 1 · 2 · 3번, 각 4인 테이블.            |
| table_sessions          | 착석/이용 세션. 테이블이 비워질 때까지 주문을 묶는 핵심 단위.           |
| party_groups            | 여러 테이블의 일행 그룹.                                 |
| orders                  | 추가 주문마다 새 주문 row. table_id가 아니라 session_id 중심. |
| order_items             | 메뉴별 수량/금액 상세. 주문합계의 원천.                        |
| checkouts               | 결제 단위. session_id와 연결.                         |
| checkout_items          | 결제와 주문항목 연결.                                   |
| payments                | 현금/카드/기타 실제 결제 기록. 취소/환불 이력 보존.                |
| staff                   | 직원과 역할, POS PIN 및 향후 관리센터 인증.                  |
| menu / options          | 메뉴, 다국어, 이미지, 추가옵션.                            |
| QR_sessions / QR_orders | 고객 QR 임시장바구니 후 승인 시 정식 주문으로 복사.                |
| kitchen_tickets         | 주방 전달/KDS.                                     |
| table_session_discounts | 세션별 할인 기록.                                     |
| customers               | 고객 기본정보. 현재 설계/저장 연결 진행 중.                     |

### 주문 연결의 핵심

테이블 선택 → table\_session 생성 → orders 누적 → order\_items 누적 → 결제 시 동일 session\_id의 미결제 범위를 checkout으로 계산한다. 테이블 이동은 세션/테이블 관계를 변경하고 주문 자체의 역사성을 훼손하지 않는다.

## 6. 직원 · 인증 정책

기존 staff 기본 필드는 staff\_id, staff\_code(unique), name, pin\_hash(nullable), role(OWNER/MANAGER/STAFF), is\_active, cancel\_requires\_pin, created\_at, updated\_at 등이다. 001

손창호 OWNER, 000 매장 공용 STAFF 등이 사용되었다.

## 최신 인증 방향

- <b>POS</b>: 화면에서 직원 이름 선택 → POS 전용 4자리 숫자 PIN. staff\_code(001 등)는 사용자가 로그인에 쓰지 않는다.
- <b>TDK 관리센터</b>: 직원 선택이 아니라 로그인 아이디 + 관리센터 전용 6자리 숫자 PIN.
- POS 4자리 PIN과 관리센터 6자리 PIN은 서로 다른 hash로 보관한다.
- 관리센터용 후보 컬럼: admin\_login\_id(unique, nullable), admin\_pin\_hash(nullable). 기존 pin\_hash는 POS용.
- staff\_id는 내부 PK이므로 유지한다. staff\_code는 현재 코드 참조 여부를 조사한 뒤 로그인 UI에서는 제거하되 DB DROP은 서두르지 않는다.
- 관리센터는 우선 OWNER만 접근. MANAGER/STAFF/000 공용 계정은 차단. 향후 권한 세분화.
- PIN은 평문 저장 금지. 기존 `script+salt` 검증 구조 재사용. 관리센터 PIN은 `^\d{6}$`, POS PIN은 `^\d{4}$`.
- 세션은 256-bit 랜덤 토큰, DB에는 SHA-256 token hash, HttpOnly cookie, SameSite=Lax, production Secure, 기본 12시간 방향.
- SHARED 모드에서는 000이 일반 영업을 담당하지만 민감 작업은 실제 직원 재인증.

## 7. POS 메인 화면

- 좌측 주문/정보, 중앙 테이블, 우측 메뉴·주문/결제 중심의 3:4:3에 가까운 비율을 유지한다.
- 24인치 1920×1080에서도 중앙/메뉴가 과도하게 늘어나지 않도록 메뉴 패널은 fixed-ish 폭을 유지하고 글자는 조금 확대한다.
- 좌측 상단 상호 '담동김치찌개', 날짜·시간, 로그인 사용자. 사용자명 클릭/로그아웃 흐름.
- 좌측 탭: 주문내역 / 주문상세 / 테이블별 / 주방전달. 주방전달은 향후 양방향.
- 중앙은 실제 매장과 유사한 테이블 배치. 빈 테이블은 회색, 착석 시 의자 나무색 등 시각 변화.
- 우측은 주문 리스트·합계·결제. 추가 주문은 같은 세션에서 합산 표시.

## 하단 메뉴 최신 방향

관리 → 판매 → 영수증 → 인원 → 할인 → 테이블관리 → 고객관리

- 한 줄 실제 DOM 버튼, 가로 스크롤, scrollbar 숨김, 터치 swipe + mouse drag.
- CSS Scroll Snap으로 놓으면 가장 가까운 버튼 슬롯에 정렬.
- 실제 hidden content를 `scrollLeft/scrollWidth/clientWidth`로 판정하여 좌/우 세로 말줄임표(...) 표시.
- 첫 화면에서는 테이블관리까지 완전히 보이고 고객관리는 오른쪽으로 스크롤하여 접근하는 구성이 목표.

## 8. 주문·테이블·일행 처리

- 테이블 선택 → 메뉴 선택 시 수량 팝업 → 우측 주문리스트 실시간 추가 → 주문 확정.
- 빈 테이블 선택만으로 인원창을 띄우지 않는다. 첫 주문 확정 시 인원 확인.
- 추가 주문은 기존 orders row를 덮어쓰지 않고 새 orders row로 누적한다.
- 일행 그룹은 여러 테이블을 묶어 그룹 주문/결제를 합산한다. 해제 후 재그룹 가능.
- 테이블 이동 시 session과 table 관계를 갱신하되 주문 이력을 유지한다.
- 테이블 관리 기능: 테이블 이동 / 일행 지정 / 일행 취소 / 합석 / 합석 분리 / 주문 복사 / 테이블 취소.
- 테이블관리는 별도 페이지가 아니라 메인 POS 위 모달로 사용하는 방향.

- 테이블 취소는 실제 직원 PIN 필요. 올바른 4번째 숫자 입력 시 실행하는 현장 흐름. 전체 취소가 아닌 item cancel은 거래 전체 취소가 되는 경우에만 PIN 정책 적용.

## 테이블 카드 표시

빈 테이블: 번호만 중앙.<br/>사용중: 좌상단 테이블 번호 / '=' 받을금액' / 선불이 있으면 '- 선불금액' / '이용시간(인원)' 순서. 선불이 없으면 해당 줄 숨김.

## 9. 인원 · 할인 · 메뉴 UX

- 인원 확인: 성인/아동. 아동이 있을 때만 총 인원을 별도 표기하는 규칙.
- 빠른 인원 1~4 버튼은 크게, +/- 는 상대적으로 작게. 확인 버튼은 넓고 낮게.
- 할인 DB: table\_session\_discounts(session\_id, type SNS\_REVIEW/AMOUNT/PERCENT, label, amount, rate, staff\_id, created\_at).
- 메뉴 카테고리: ALL / 찌개 / 단품 / 음료 / 추가 / 포장.
- 새 테이블 선택 시 메뉴 카테고리를 ALL로 초기화.
- 메뉴 패널 약 9행×2열 가시 목표. 메뉴명은 중앙 정렬, 장바구니 아이콘은 이미지 우하단.
- 화폐 표시는 기본적으로 '원'을 붙이지 않는다.

## 10. 결제 핵심 구조

- 기본 결제는 카드/현금/기타결제/고객결제 방향. 과거 '선불' 버튼은 '고객결제'로 이름을 바꾸고 별도 '후불' 버튼은 제거하는 방향.
- 결제 버튼을 누르면 결제가 실제 저장된다. 결제 모달은 결제 후에도 남고 받을금액 0일 때만 '결제완료' 활성화.
- 뒤로가기로 결제 모달을 닫아도 이미 저장된 결제는 선불/부분결제로 남는다.
- 현금 초과수납: tendered는 받은 현금, applied는 받을금액까지만 매출 적용, change는 거스름. 예 24,000에 30,000 입력 → 적용 24,000, 거스름 6,000.
- 카드 초과결제는 차단.
- party checkout은 서버가 일행 전체 범위를 계산해야 한다.
- 결제취소는 DELETE가 아니라 reversal/status 이력. 완료 거래의 취소는 실제 직원 재인증.
- 향후 카드 VAN 연동 시 VAN 취소 성공 후 DB 상태 변경.

## 결제 요약 표시

주문금액 → 할인 → 결제금액 → 결제내역(선불현금/선불카드 등) → 받을금액. 할인은 빨강, 선불/받을금액은 파랑 계열. 할인 없으면 불필요한 할인 줄은 숨김.

## 11. 기타결제

- 공통 숫자 입력 컴포넌트 NumericInputKeypad: 7 8 9 / 4 5 6 / 1 2 3 / 000 0 C. 키패드는 순수 숫자값만 담당.
- 기타결제는 AMOUNT 또는 QUANTITY 방식. AMOUNT 선택 시 현재 받을금액 자동 입력, 첫 숫자 입력이 기본값을 교체, C는 0.
- 쿠폰/상품권/식권 등은 결제수단 설정으로 관리할 예정.
- 잔액 정책 예: CASH\_CHANGE 또는 FORFEIT. 쿠폰 10,000×3으로 28,000 결제 시 submitted 30,000 / applied 28,000 / forfeited 2,000을 모두 기록할 수 있어야 한다.
- 관리센터의 '결제관리'에서 기타결제 관리로 확장한다.

## 12. 고객 및 고객결제 설계

고객은 '선불 고객/후불 고객'으로 미리 분류하지 않는다. 한 고객에게 선불금 입금 · 사용과 미수금 발생 · 수납이 모두 일어날 수 있다.

| 필드                      | 의미                                 |
|-------------------------|------------------------------------|
| name                    | 이름 또는 업체명. 필수. 예: 홍길동 / 제주상사       |
| contact_name            | 담당자. 업체 고객용, 선택.                   |
| phone                   | 휴대전화 또는 일반전화, 선택.                  |
| email                   | 이메일, 선택.                           |
| memo                    | 지속적인 고객 메모. 예: 매월 말 정산. 주문 요청과 별개. |
| is_payment_managed      | 선불금 · 미수금 등 고객별 결제관리 대상 여부.        |
| is_active               | 삭제 대신 사용/사용중지.                     |
| created_at / updated_at | 생성/수정 시간.                          |

- customers에는 prepaid\_balance나 receivable\_balance 같은 금액을 단순 저장하지 않는 방향. 거래 원장으로 입금/사용/미수/수납을 기록하고 필요하면 캐시 잔액을 트랜잭션으로 유지한다.
- 관리센터 고객등록 UI는 이름/담당/전화/이메일/메모/결제관리 체크로 설계되었다.
- 현재 고객 저장은 '고객을 저장하지 못했습니다' 오류가 발생한 상태였고, customers 실제 테이블 존재/컬럼 일치/API INSERT를 점검하는 단계다. 따라서 이 부분은 완성 상태로 간주하면 안 된다.
- POS 고객관리에서는 상세 입력보다 고객 검색/선택을 우선한다. 고객등록은 PC 관리센터가 기본이며 POS 간편등록은 필요할 때 추가한다.
- 손님 요청(덜 맵게 등)은 고객 memo가 아니라 현재 table\_session/order에 속하는 별도 데이터다.

## 13. TDK 관리센터

- 경로 /admin. 같은 tdk - pos 프로젝트 내부에 존재하며 PC용 UI다.
- 대시보드와 좌측 사이드바 구조. 고객관리(/admin/customers), 결제관리(/admin/payments)를 우선 추가하는 방향.
- 고객관리: 검색 + 고객등록 + 목록. 목록은 이름/담당/전화/이메일/결제관리/상태 중심.
- 고객등록은 모달. PC 키보드 사용, 별도 화면 한글 키보드 없음.
- 결제관리: 고객결제 관리 / 기타결제 관리의 상위 진입 화면부터 만들고 세부 기능은 단계적으로 추가.
- 향후 메뉴: 매출관리, 거래관리, 메뉴관리, 재고관리, 매입관리, 직원관리, 현금시재, 매장설정, 시스템관리.
- POS에 이미 있는 관리 기능은 PC 관리센터에서 실제로 대체 완료된 뒤 POS 진입 UI만 제거한다. DB/API까지 지우지 않는다.

### 관리센터 로그인 최신 결정

- /admin/login에서 '아이디 + 6자리 PIN' 사용. 직원 선택 UI를 사용하지 않는다.
- OWNER만 우선 허용. admin\_login\_id + admin\_pin\_hash를 staff에 추가하는 방향.
- POS의 4자리 pin\_hash와 관리센터 6자리 admin\_pin\_hash는 독립.
- POS OWNER 세션만으로 관리센터에 자동 진입시키지 않는다. 관리센터 인증 컨텍스트를 서버에서 구분.
- 관리센터 전체 /admin/\* 보호, /admin/login 제외. 직접 URL 접근도 로그인으로 보낸다.
- 상단에 실제 직원명/OWNER 및 로그아웃. 로그아웃은 서버 세션까지 종료.

- 관리 API도 화면 보호만 믿지 말고 서버 권한 검증. 로그인 실패 횟수 제한/지연과 HTTPS 고려.

## 14. 판매내역 · 거래상세 · 결제 변경

- 판매내역 기간: 전월 / 전주 / 어제 / 오늘 / 기간지정. 오늘 기본.
- 컬럼: 주문시간, 테이블, 메뉴, 결제금액, 결제수단, 거래상태.
- 주문시간은 최초 주문시간을 사용하고 결제시간과 구분한다.
- 거래 상세: 테이블/고객/주문시간 · 거래시간/거래번호, 주문금액 · 할인금액 · 결제금액, 주문내역, 취소내역, 결제내역, 영수증 재출력.
- 완료 거래 결제수단 변경: 원결제 선택 → 실제 직원 PIN → 원결제 취소/환불 → 성공 → 새 결제 → 완료.
- 판매일은 원래 날짜를 유지하고 변경/환불은 실제 처리 날짜에 별도 기록. 예: 전일 카드 +40k, 다음날 카드취소 -40k + 새 카드 +40k.
- 현재 payments에는 method\_type, checkout\_id, payment\_method\_id, amount, status, paid\_at, cancelled\_at 등이 있고 original\_payment\_id/REFUNDED 개념은 있으나 완료거래 환불 흐름은 아직 미완성.

## 15. 현금시재

- 일일 시작금액은 수동 입력 또는 전일 최종 예상금액 복사 중 하나. 전일 데이터를 수정하지 않고 새 날짜 opening으로 복사.
- 예상 시재 = opening + 유효 현금결제 net + manual IN - manual OUT.
- 권장 테이블: cash\_daily\_sessions, cash\_manual\_movements, cash\_movement\_categories.
- SHARED 모드의 민감 시재 작업은 실제 직원 PIN 재인증을 고려.
- 현재는 환불/기타결제/고객결제 구조가 정리된 뒤 구현하는 것으로 보류.

## 16. 영수증 · 프린터 · Device Agent

- BIXOLON SRP - 350III USB. 현금서랍은 프린터에 연결되어 기존 POS에서 동작 확인.
- 웹브라우저가 USB 프린터를 직접 제어하지 않고 C#/.NET 8 Device Agent가 localhost에서 장치를 제어.
- 흐름: Next.js → localhost Device Agent → Windows RAW/ESC - POS → 프린터/금고.
- Agent 개발 실행: dotnet run - -project .\tdk-device-agent\tdk-device-agent.csproj - -launch-profile http
- 기본 localhost 포트 5168. 충돌 시 netstat/tasklist로 해당 PID만 종료하고 모든 dotnet 프로세스를 일괄 종료하지 않는다.
- 한글 출력/정렬/굵게는 동작 확인.
- 영수증 표기: RNo.=receipt, TNo.=table. [재출력] 뒤 여백, '결제금액' 대신 '받을금액' 방향, 절단 전 충분한 feed.
- 향후 현금 결제/현금 포함 혼합결제에서 금고 자동 열기, 카드-only는 열지 않기. 수동 금고열기 버튼도 고려.
- 로고는 설정에서 선택하고 영수증 최상단에 출력.

## 17. 주방/KDS · QR · 재고 확장

- KDS는 같은 tdk-pos의 /kitchen. 공유 DB/API, 초기에는 2~3초 polling. 상태는 item/batch 수준을 고려.
- QR 주문: 테이블 QR → QR\_sessions/QR\_orders 임시장바구니 → 직원 승인 시 orders로 복사. 외부 대기 QR 세션도 입장 후 테이블 QR과 연결 가능.
- QR 임시 데이터는 만료/정리 정책 필요.

- 재고: 공기밥 등 단순 수량 실시간 잔량, 주류/음료 매입/재고, 식재료 g 단위 recipe를 통한 일일 판매량 · 원가 집계.
- 메뉴는 POS용/QR용 이미지 크기 분리, 다국어 테이블, 옵션 구조를 지원하는 방향.
- 공기밥은 1인 1개 제공. 영수증에는 0원 품목으로 분리할지 메뉴에 포함할지는 운영 정책에 따라 결정.

## 18. NAS 중앙서버 · 배포

- 기본 운영 서버는 NAS로 한다. 현재 ASUSTOR AS5404T에서 MariaDB가 이미 동작하며 Docker로 Home Assistant와 ntfy도 운영 중.
- 가족용 미디어 서버와 TDK - POS를 같은 NAS에서 운영하는 것은 일반적인 Direct Play 위주라면 충분히 현실적. 4K 실시간 트랜스코딩이 많으면 CPU 부하를 별도 관찰.
- 성능보다 장애 분리가 더 중요한 리스크. NAS 재부팅/장애 시 POS도 영향을 받는다. 향후 필요하면 POS 전용 NAS로 분리.
- MariaDB를 굳이 Docker로 옮기지 않는다. 기존 NAS MariaDB 유지 + TDK - POS Next.js Docker 연결.
- POS PC와 업무 PC는 브라우저로 NAS 중앙서버에 접속. 프린터/금고/카드단말기는 POS PC의 로컬 Device Agent.
- 외부 인터넷에서 /admin을 노출할 경우 HTTPS, 로그인 rate limit, 방화벽/VPN/접근정책을 반드시 검토.

## 19. 공통 UI 규칙

- 터치 POS는 큰 글자 · 높은 대비 · 충분한 터치 영역. 사용자의 근거리 가독성을 고려하여 지나치게 작은 글자를 피한다.
- POS 하위창/모달 공통 헤더: #455A64, 흰색 굵은 제목, 왼쪽 흰 원형 back 버튼 + #455A64 화살표.
- 진짜 하위창은 '부모 > 현재창' breadcrumb 표시. 부모는 작고 열게, 현재창은 크고 굵게. 작은 확인/오류/PIN 다이얼로그는 제외.
- 브라우저 alert 대신 자체 오류/확인 UI.
- 가로 선택 스크롤러는 hidden scrollbar + touch swipe + mouse drag + Scroll Snap + 실제 hidden 여부 기반 ... 표시.
- 결제/키패드/직원선택 등 공통 UI는 시각 컴포넌트를 재사용하되 비즈니스 인증 로직까지 억지로 합치지 않는다.
- 통화 단위 '원'은 기본적으로 생략.

## 20. 현재 완료/진행/미완성

| 상태       | 내용                                                                                                                                |
|----------|-----------------------------------------------------------------------------------------------------------------------------------|
| 동작/진행 확인 | DB 연결, /api/db - test, /api/tables, 테이블 클릭/선택, 주문 입력 · 합산, 기본 POS 레이아웃, 로그인/로그아웃 UI, 메뉴/테이블/결제 관련 상단수 UI, 프린터 Device Agent 한글 출력. |
| 최근 구현 중  | /admin 관리센터 기본, 고객관리 UI, 고객등록 모달, 결제관리 기본 화면.                                                                                     |
| 현재 문제    | 고객등록 시 '고객을 저장하지 못했습니다.' 오류. customers 실제 DDL/API/schema 일치 여부를 확인하고 해결해야 함.                                                      |
| 최신 설계 변경 | 관리센터 로그인은 직원 선택+4PIN이 아니라 admin ID + 6자리 PIN. POS는 이름 선택 + 4자리 PIN 유지.                                                            |
| 미완성      | 고객 거래원장/선불/미수금, 완료거래 결제변경/환불, 현금시제, 실제 VAN 연동, KDS, QR 정식 흐름, 재고/원가, 관리센터 전체 기능.                                                  |

## 21. 처음부터 다시 만드는 권장 순서

1. 프로젝트 생성: Next.js + TypeScript + Tailwind + Drizzle + mysql2. 환경변수로 NAS MariaDB TDK\_POS 연결.
2. DB 연결 테스트(/api/db - test)와 dining\_tables 조회(/api/tables)부터 확인.
3. 핵심 DB: dining\_tables → table\_sessions → party\_groups → orders → order\_items → checkout/payments → staff 순으로 확정.
4. 직원 POS 인증: 이름 선택 + 4자리 PIN, PERSONAL/SHARED, 000 공용계정 정책.
5. POS 메인 3영역 레이아웃과 테이블 선택/메뉴/주문 누적.
6. 인원 확인, 할인, 일행/합석/이동/취소 등 table session 기능.
7. 결제: 현금/카드/기타, 부분결제, 현금 거스름, 결제완료 상태.
8. Device Agent + BIXOLON 영수증 출력. 이후 금고열기.
9. 판매내역/거래상세/취소 이력.
10. /admin 관리센터 공통 layout.
11. 관리센터 인증: admin\_login\_id + 6자리 admin PIN, OWNER - only부터 시작.
12. customers DDL/API를 확정하고 고객등록 저장 오류 없이 CRUD/검색을 완성.
13. customer account transaction 원장을 설계하여 고객결제(선불/미수)를 연결.
14. 기타결제 설정을 관리센터로 이동.
15. 완료거래 결제수단 변경/환불, 현금시재.
16. 그 후 메뉴/재고/매입/직원/설정, KDS, QR을 단계적으로 추가.
17. 검증된 빌드를 NAS Docker로 배포하고 /pos, /admin, 향후 /kitchen을 중앙 운영.

## 22. 검증 체크리스트

- DB: 각 schema와 실제 MariaDB DDL 일치, FK/PK/unique/default/nullable 확인.
- 주문: 추가주문이 덮어쓰기 없이 누적되고 session 합계가 맞는지.
- 일행: 그룹 전체 조회/결제 범위가 서버에서 계산되는지.
- 결제: 부분결제, 현금 초과수납/거스름, 카드 초과차단, 취소 이력 보존.
- 인증: POS 4PIN과 admin 6PIN 분리, hash만 저장, 000의 관리센터 접근 차단.
- 관리센터: /admin/\* 직접 URL 보호, API도 서버 권한 검증.
- 고객: 이름/담당/전화/이메일/메모/결제관리 저장 및 검색, 잔액은 원장으로 관리.
- 영수증: 한글/정렬/재출력/절단/feed 및 Device Agent 연결.
- UI: 1920×1080 POS에서 잘림 없음, 터치 영역 충분, 가로 스크롤/…/snap 정상.
- 배포: .env 비공개, Docker 재시작 정책, NAS 백업, DB 백업, 장애 시 복구 절차.

## 23. 주요 결정사항 요약

- TDK - POS는 NAS 중앙서버 기반의 하나의 웹 시스템으로 간다.
- POS와 PC 관리센터는 같은 코드/DB/API를 쓰되 UI와 역할은 분리한다.
- POS는 영업 기능만 남기고 상세 관리는 PC 관리센터로 점진적으로 이동한다.
- POS 로그인은 직원 선택 + 4자리 PIN, 관리센터는 아이디 + 6자리 PIN.

- 고객은 개인/업체를 하나의 customers로 관리하고 담당자 필드를 둔다.
- 고객을 선불/후불 타입으로 고정하지 않는다. 금전 변화는 거래원장으로 남긴다.
- 결제와 취소는 삭제가 아니라 감사 가능한 기록으로 남긴다.
- 프린터/금고/카드단말기는 중앙 NAS가 아니라 POS PC의 Device Agent가 제어한다.
- 현재 가장 먼저 해결할 기술 과제는 customers 실제 DB 구조와 API를 맞춰 고객 저장을 정상화하는 것이다.

문서 종료 - TDK POS / TDK 관리센터 기준서 2026 - 10 - 01